

gen_server and gen_statem — A Readable Companion to the OTP Docs

Chapter 5 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why behaviours exist

A behaviour splits a process into a generic part that OTP writes once and a specific part you write as callbacks. `gen_server` and `gen_statem` are the two you will use most. The generic part is where the hard things live: the start handshake with the supervisor, system messages, debugging and tracing through `sys`, error reports, code upgrade, and the shutdown protocol.

A hand-written `receive` loop gets none of that. It looks simpler, but it cannot be suspended, inspected, traced or upgraded like the rest of the system. The rule of thumb is short: if a process lives longer than one task, make it a behaviour.

Both behaviours are still just one process. Every callback runs in sequence inside it, so a behaviour is also a serialisation point: whatever one callback is doing, every other request waits.

In Elixir, `GenServer` wraps `gen_server`. `:gen_statem` is usually used directly from Elixir; the concepts below carry over unchanged.

2. The building blocks

gen_server callbacks

Callback	Triggered by	Typical return
<code>init/1</code>	<code>start_link</code>	<code>{ok, State}</code> or <code>{ok, State, {continue, Term}}</code>
<code>handle_call/3</code>	<code>gen_server:call</code> (caller waits)	<code>{reply, Reply, State}</code> or <code>{noreply, State}</code>
<code>handle_cast/2</code>	<code>gen_server:cast</code> (caller does not wait)	<code>{noreply, State}</code>
<code>handle_info/2</code>	Any other message: timers, <code>'DOWN'</code> , <code>'EXIT'</code>	<code>{noreply, State}</code>
<code>handle_continue/2</code>	A <code>{continue, Term}</code> return	<code>{noreply, State}</code>
<code>terminate/2</code>	Stop, or shutdown when trapping exits	ignored

If you don't export `handle_info/2`, the default logs unexpected messages and drops them.

gen_statem in one table

A `gen_statem` separates the *state* (which mode the machine is in) from the *data* (everything else). Events arrive with a type:

Event type	Comes from
<code>{call, From}</code>	<code>gen_statem:call</code>
<code>cast</code>	<code>gen_statem:cast</code>
<code>info</code>	Ordinary messages
<code>internal</code>	Your own <code>{next_event, internal, E}</code> action
<code>state_timeout</code>	A timer that dies on any state change
<code>{timeout, Name}</code>	A named timer that survives state changes
<code>timeout</code>	An event timer that dies on <i>any</i> event

There are two callback modes:

- `state_functions`: one function per state, so states must be atoms. Easy to read when states are few and simple.
- `handle_event_function`: one `handle_event/4` for everything, so states can be any term, such as `{connected, Retries}`.

The actions that make `gen_statem` worth using:

- **postpone**: put this event back and retry it after the next state change.
- **next_event**: inject an event to be handled before anything else in the mailbox.
- **state enter calls**: run code on every entry into a state, in one place instead of at every transition.

Which one?

Use <code>gen_server</code> when	Use <code>gen_statem</code> when
The process is a service: a cache, a pool, a registry	Behaviour depends on a mode: connecting, connected, backing off
Every request is handled the same way whatever the state	Some events are only valid in some states
Timeouts are simple or absent	You need per-state timers or events that wait for the right state

The docs put the overhead of `gen_statem` over `gen_server` as marginal, so choose by shape, not speed.

3. How the runtime actually does it

A call is a monitored request

`gen_server:call` is the pattern from chapter 2: monitor the server, send a request tagged with a reference, and wait for a reply matching it. The default timeout is 5000 ms. If the server dies, the caller exits with the server's reason (or `noproc` if it was never there). Since OTP 24 an alias ensures that a reply arriving after the timeout is discarded.

Replies do not have to be immediate

`handle_call` may return `{noreply, State}` and answer later with `gen_server:reply(From, Reply)` from any later callback, or from another process. This is how a server stays responsive while slow work happens elsewhere.

Timeouts in `gen_server` are crude

`{noreply, State, Timeout}` sends timeout to `handle_info` only if *no message at all* arrives for Timeout ms. Any message cancels it, and the docs admit that even a system message restarts it ("a known and unfortunate flaw"). For real timers, use `erlang:send_after/3` or `erlang:start_timer/3` and keep the reference.

Timeouts in `gen_statem` are precise

Timer	Cancelled by	Good for
<code>timeout</code> (event)	Any event	"Nothing at all happened for N ms"
<code>state_timeout</code>	Leaving the state	"Give up on connecting after 10 s"
<code>{timeout, Name}</code>	Only by you (or it fires)	Timers that span states, several at once

How postpone works

A postponed event is set aside and retried after the next *state change*, meaning `NewState /= OldState`. Changing only the data does not retry anything. The docs spell out the consequence: if a value decides which events you can handle, it belongs in the state, not the data.

4. Plausible but wrong

4.1 Slow work inside `handle_call`

```
handle_call({report, Id}, _From, State) ->
    Report = build_report(Id),           %% 20 seconds
    {reply, Report, State}.
```

For those 20 seconds every other caller is stuck, and anyone using the default 5000 ms timeout crashes with `timeout`. The server looks hung while it is actually busy. Fix: hand the work to a separate process and reply from there:

```
handle_call({report, Id}, From, State) ->
    spawn_link(fun() -> gen_server:reply(From, build_report(Id)) end),
    {noreply, State}.
```

In production, start the worker under a task supervisor instead of `spawn_link` (chapter 2, section 4.7).

4.2 An idle timeout that never fires

```
handle_info(tick, State) -> {noreply, State, 60000};
handle_info(timeout, State) -> {stop, normal, State}.
%% a metrics collector sends 'tick' every 10 s
```

Every message restarts the 60-second timeout, so a process meant to stop after a minute of real inactivity lives forever. Fix: track the time of the last meaningful event and use an explicit timer (`erlang:start_timer/3`), or use `gen_statem` with a named timeout that only meaningful events reset.

4.3 A state machine hidden in a gen_server

```
handle_call({send, Msg}, _From, #{status := connecting} = S) ->
    {reply, ok, S#{pending := [Msg | maps:get(pending, S)]}};
handle_info(connected, #{pending := P} = S) ->
    [do_send(M) || M <- lists:reverse(P)],
    {noreply, S#{status := connected, pending := []}}.
```

Each status adds more clauses, manual queues and chances to lose a message on a path you forgot. Fix: make it a `gen_statem` where `send` is postponed in `connecting` and replayed automatically on entering `connected`.

4.4 Postponing on data, not state

```
handle_event({call, _}, {send, _}, open, #{credit := 0}) ->
    {keep_state_and_data, [postpone]};
handle_event(info, {credit, N}, open, D) ->
    {keep_state, D#{credit := N}}.
```

The state stays `open`, so the postponed sends are never retried, even after credit arrives. The callers hang until they time out. Fix: make credit part of the state, such as `{open, Credit}` in `handle_event_function` mode, or move between states `open` and `blocked`.

4.5 Using an event timeout for a per-state limit

```
connecting(enter, _Old, D) ->
    {keep_state, D, [{timeout, 10000, give_up}]};
connecting(info, {status_query, From}, D) ->
    From ! connecting,
    keep_state_and_data.
```

An event timeout is cancelled by *any* event, so the first status query cancels it. Under monitoring that polls every few seconds, the connection attempt never gives up. Fix: `{state_timeout, 10000, give_up}`, which only a change of state cancels.

4.6 Two servers calling each other

```
%% in a:handle_call(get_total, ...)
Total = gen_server:call(b, sum),
%% in b:handle_call(sum, ...)
Rate = gen_server:call(a, rate),
```

A waits for B, B waits for A. Nothing moves until the 5-second timeout, then one or both crash. It only happens when the two requests overlap, so it rarely shows up in tests. Fix: make one direction a `cast` or a deferred reply, or restructure so data flows one way only.

4.7 Casting without backpressure

```
log(Line) -> gen_server:cast(log_writer, {line, Line}).
```

When producers outpace the writer, its mailbox grows without limit and memory climbs until the node dies. A `call` naturally slows producers to the writer's speed. Fix: use `call` for high-volume paths, or add explicit batching or load shedding.

5. Review checklist and sources

- ☐ Long-lived processes are behaviours, not hand-written receive loops
- ☐ No callback does slow or blocking work inline; slow work replies later with `gen_server:reply/2`
- ☐ Real timers use `send_after/start_timer`, or the right `gen_statem` timeout type
- ☐ Anything whose handling depends on a mode is a `gen_statem`
- ☐ Values that decide which events can be handled live in the state, not the data
- ☐ No two servers `call` each other
- ☐ High-volume producers have backpressure (`call`, batching or shedding)
- ☐ Servers that need cleanup trap exits (chapter 1, section 4.1)

Sources

- `gen_server` Behaviour, OTP Design Principles:
https://www.erlang.org/doc/system/gen_server_concepts.html

- gen_server module reference: https://www.erlang.org/doc/apps/stdlib/gen_server.html
- gen_statem Behaviour, OTP Design Principles: <https://www.erlang.org/doc/system/statem.html>